Software Requirements Specification (SRS) for FLARE

Document ID: FLARE-SRS-2026-V1 **Project Name:** FLARE (Offline-First, Edge-Resilient Application Architecture) **Prepared For:** BlueRhea Technologies **Prepared By:** System Architecture Team **Date:** 2026-10-05 **Version:** 1.0 **Classification:** Internal Technical Specification **Pages:** ~200 (approximate structure)

Revision History

Version	Date	Author	Description
0.1	2026-10-05	FLARE Team	Initial Draft
1.0	2026-10-05	FLARE Team	Final Release

Table of Contents

1. Introduction
2. Overall Description
3. External Interface Requirements
4. System Features
5. Other Nonfunctional Requirements
6. Other Requirements
7. Appendices

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the complete functional, non-functional, interface, and operational requirements for FLARE - an enterprise-grade, offline-first application architecture, local data synchronization engine, and edge intelligence daemon.

FLARE is designed for environments with intermittent, unreliable, or untrusted network connectivity, including regional educational systems, mobile money micro-services, local enterprise nodes, and field-deployed edge devices.

1.2 Scope

FLARE provides:

- Offline-first data persistence with local SQLite (WAL mode) as the source of truth
- Local-first application runtime with optimistic UI via IndexedDB/Dexie.js
- Background daemon (`flared`) for orchestration, scheduling, and sync
- Natural Git commit engine for human-like local commit history
- Cryptographic node-locking (Ed25519), zero-trust authentication, and payload encryption (AES-256-GCM)
- Asynchronous sync with exponential backoff when connectivity is available
- Unix domain socket IPC with optional HTTPS bridge for local clients
- Cross-platform frontend (React + TypeScript + Vite + Tailwind CSS)
- Production-ready operations via systemd, encrypted backups, and observability

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
FLARE	Offline-First, Edge-Resilient Application Architecture
flared	FLARE daemon (background service)
IPC	Inter-Process Communication
WAL	Write-Ahead Logging (SQLite)
CRDT	Conflict-Free Replicated Data Type
Ed25519	Edwards-curve digital signature algorithm
AES-256-GCM	Advanced Encryption Standard with Galois/Counter Mode
mTLS	Mutual Transport Layer Security
SRS	Software Requirements Specification

1.4 References

- FLARE Technical Specification (Flare.md)
- SQLite WAL Documentation
- Ed25519 (RFC 8032)
- NIST AES-GCM
- Dexie.js Documentation
- FastAPI/Uvicorn Documentation

1.5 Overview

This document follows IEEE-style SRS structure covering system purpose, constraints, interfaces, detailed features, and quality attributes.

2. Overall Description

2.1 Product Perspective

FLARE is a self-contained, local-first system composed of multiple integrated components:

- **Frontend Layer**: React/Vite/TypeScript/Tailwind with Dexie.js for offline IndexedDB
- **Daemon Layer** (`flared`): Go-based background service managing state, licensing, sync, Git
- **Crypto Layer**: C++ shared library for Ed25519, AES-256-GCM, hardware fingerprinting
- **API Layer**: Django REST API for management, audit, and sync queue access
- **Bridge Layer**: FastAPI HTTPS bridge providing secure local access to IPC
- **Data Layer**: SQLite (WAL) as local authority; optional remote replication targets

2.2 Product Functions

- Deterministic local state management with offline authority
- Natural Git commit generation with time-based format rotation
- Node-locked license verification and hardware fingerprinting
- Offline sync queue with retry, backpressure, and exponential backoff
- Secure IPC over Unix domain socket (loopback-only bridge)
- Encrypted local storage awareness and backup/restore
- Audit logging with hash-chain integrity
- Systemd integration for production resilience

2.3 User Classes and Characteristics

Class	Characteristics
System Operator	Deploys, monitors, manages services via CLI/systemd
Developer	Extends/maintains codebase, runs local dev/test
End User	Interacts with FLARE UI in offline/online modes
Admin	Manages node config, audit logs via Django Admin

2.4 Operating Environment

- **Platform**: Linux (primary target; design supports cross-platform)
- **Runtime**: Go 1.20+, Python 3.10+, Node 18+, C++17
- **Services**: systemd (production), Unix domain sockets
- **Storage**: Local filesystem (~/.flare-data), SQLite WAL

- **Network**: Optional (loopback-only for local clients; outbound only on sync)

2.5 Design and Implementation Constraints

- Offline-first: core functionality must work without internet
- No hard dependency on external cloud during normal operation
- Use local cryptographic identity (Ed25519 keypair)
- Deterministic, reproducible builds
- Secure defaults: loopback-only, token-based auth for bridge
- Self-contained deployment with minimal external deps

2.6 Assumptions and Dependencies

- Local filesystem writable for data dir and logs
- systemd available for production service management
- OpenSSL available for bridge TLS cert generation
- Git installed if Natural Commit Engine used
- SQLite3 available for maintenance/backup

2.7 Apportioning of Requirements

Future releases may extend to P2P sync, multi-node clustering, and additional platform support.

3. External Interface Requirements

3.1 User Interfaces

- **Web UI (Frontend)**: Vite/React SPA at http://127.0.0.1:5173 (dev) or http://localhost:8080 (Docker)
- **Dashboard**: System status, daemon connectivity, offline queue metrics, Dexie item counts
- **About Page**: Product branding (FLARE) + Partner branding (BlueRhea Technologies) with theme-aware logos
- **Django Admin**: http://127.0.0.1:8000/admin/ for node config, audit logs, sync queue

3.2 Hardware Interfaces

- Local storage (filesystem)
- Network interface (optional, outbound for sync when online)

3.3 Software Interfaces

Interface	Protocol	Purpose
Unix Domain Socket	JSON line-based	flared IPC at /tmp/flared.sock
HTTPS Bridge	REST/JSON over TLS	Local loopback access to IPC (Bearer token)
Django REST API	HTTP/JSON	/api/health, /api/sync/queue, /api/sync/pull
Git CLI	subprocess	Natural commit operations
SQLite	WAL	Local persistence

3.4 Communications Interfaces

- Unix socket (AF_UNIX) for local IPC
- HTTPS (TLS 1.2+) for bridge on 127.0.0.1:8443
- HTTP for Django/API and Vite dev (loopback only)
- Optional WebSocket (WSS) for remote sync (per spec)

4. System Features

4.1 Offline-First Data Persistence

Description: Local SQLite database with WAL mode serves as immutable local authority.

- FR-1: All reads/writes must succeed offline
- FR-2: WAL mode enabled with synchronous=NORMAL
- FR-3: Foreign keys enforced
- FR-4: Automatic recovery on restart
- FR-5: Atomic snapshots via backup script

4.2 flared Daemon (Background Engine)

Description: Persistent background service orchestrating core logic.

- FR-6: Auto-start on boot via systemd
- FR-7: Graceful shutdown on SIGTERM/SIGINT
- FR-8: Health/status reporting over IPC
- FR-9: Background sync worker with periodic ticks
- FR-10: Robust restart on failure

4.3 Natural Git Commit Engine

Description: Generates human-like commit messages with time-based rotation.

- FR-11: Detect unstaged/staged changes via git status
- FR-12: Stage changes (git add -A)
- FR-13: Rotate formats every 2 hours: CASUAL*SHORT, CONVENTIONAL, TWO*LINE
- FR-14: Avoid automation/tool markers
- FR-15: Two-line commits reference last commit context
- FR-16: Local commits succeed offline

4.4 Offline Sync Queue

Description: Durable queue for pending sync operations.

- FR-17: Persist queue items to SQLite
- FR-18: States: PENDING, PROCESSING, COMPLETED, FAILED
- FR-19: Retry with exponential backoff + jitter
- FR-20: Max attempts configurable
- FR-21: IPC endpoints: enqueue, pull_queue
- FR-22: Idempotent processing semantics

4.5 Cryptography & Node-Locking

Description: Ed25519 + AES-256-GCM with hardware fingerprinting.

- FR-23: Ed25519 keypair generation
- FR-24: Message signing/verification
- FR-25: AES-256-GCM encrypt/decrypt
- FR-26: Hardware ID derivation
- FR-27: Node-locked license verification
- FR-28: Audit log integrity via hash chaining

4.6 Secure IPC & HTTPS Bridge

Description: Local-only secure access.

- FR-29: Unix socket for local processes
- FR-30: HTTPS bridge on 127.0.0.1 only
- FR-31: Bearer token authentication
- FR-32: CORS restricted to localhost origins
- FR-33: IP/Trusted-host allowlists
- FR-34: Rate limiting per IP
- FR-35: Security headers (HSTS, CSP, etc.)

- FR-36: Timeouts, body/line limits, concurrency caps
- FR-37: Cmd allowlist enforcement
- FR-38: /health, /ready, /metrics, /branding endpoints

4.7 Frontend (React + Dexie)

Description: Offline-capable UI with branding.

- FR-39: IndexedDB via Dexie.js
- FR-40: Optimistic UI updates
- FR-41: Theme-aware FLARE + BlueRhea logos
- FR-42: Dashboard and About views
- FR-43: IPC client with HTTP fallback
- FR-44: Responsive Tailwind UI

4.8 Django API & Admin

Description: Management interface.

- FR-45: Models: system*node*config, local*audit*logs, offline*sync*queue
- FR-46: REST endpoints for health and sync
- FR-47: Branded Django Admin
- FR-48: Static logo assets served

4.9 Operations & Backup

Description: Production readiness.

- FR-49: systemd service units (hardened)
- FR-50: WAL-safe backup script
- FR-51: TLS cert generation for bridge
- FR-52: Docker compose for containerized testing
- FR-53: Local multi-process runner
- FR-54: Makefile targets

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- NFR-1: Daemon startup < 1s
- NFR-2: IPC roundtrip < 10ms (Unix socket)
- NFR-3: UI initial load < 2s (local dev)
- NFR-4: Sync queue processing non-blocking
- NFR-5: Concurrent IPC requests up to configurable limit

5.2 Safety Requirements

- NFR-6: Atomic SQLite writes via transactions
- NFR-7: Graceful degradation when remote unavailable
- NFR-8: No data loss on crash (WAL recovery)

5.3 Security Requirements

- NFR-9: Bridge bound to 127.0.0.1 only
- NFR-10: Token-based auth for bridge IPC endpoints
- NFR-11: Security headers enabled
- NFR-12: Rate limiting enforced
- NFR-13: Input validation on IPC commands (allowlist)
- NFR-14: TLS for bridge
- NFR-15: No secrets in logs
- NFR-16: Local-first identity (no forced cloud auth)

5.4 Software Quality Attributes

- NFR-17: Modularity (decoupled layers)
- NFR-18: Testability (local testability)
- NFR-19: Maintainability (clear structure)
- NFR-20: Portability (POSIX/Linux primary)
- NFR-21: Reliability (auto-restart, offline-first)

5.5 Business Rules

- BR-1: Local data is source of truth
- BR-2: Remote is async replication target
- BR-3: Natural commit history preserved
- BR-4: Branding elements preserved as specified

6. Other Requirements

6.1 Database Requirements

- Local SQLite with WAL mode
- Schema: system*node*config, local*audit*logs, offline*sync*queue
- Indexes on lookup fields
- Foreign key constraints enabled

6.2 Backup and Recovery

- WAL-safe snapshot via flare-backup.sh

- Package DB + keys + config
- Timestamped backups
- Offline operation

6.3 Branding Requirements

- FLARE logos (black/white) theme-aware
- BlueRhea Technologies logo in reserved About slot (SVG)
- Logos distributed to all UI/API/bridge/static paths
- Django Admin branded

6.4 Deployment Requirements

- systemd units for flared and bridge
- Docker compose for local testing
- Local multi-process runner script
- Makefile for common tasks

6.5 Documentation Requirements

- Technical spec (Flare.md)
- README with full setup
- BUILD.md
- Bridge README
- This SRS document

7. Appendices

7.1 Appendix A: IPC Command Set

Command	Params	Response
ping	{}	{pong: timestamp}
status	{}	{engine, socket, storage, branding}
health	{}	{status}
enqueue	{entity_type, payload}	{ok}
pull_queue	{limit}	{items[]}

7.2 Appendix B: Data Model

See Django models and SQLite DDL in technical specification.

7.3 Appendix C: Service Endpoints

- Frontend: http://127.0.0.1:5173
- Django: http://127.0.0.1:8000
- Bridge: https://127.0.0.1:8443
- Flared: /tmp/flared.sock

7.4 Appendix D: Natural Commit Format Examples

- CASUAL_SHORT: "update 3 project files and fix edge cases"
- CONVENTIONAL: "refactor(core): optimize working directory assets (3 files)"
- TWO_LINE: "refactor local module interfaces" + blank line + body

7.5 Appendix E: Glossary of Key Terms

Offline-first, local-first, zero-trust, CRDT, node-locking, WAL, IPC.